



TEORÍA DE ALGORITMOS (TB024/75.29)

Trabajo Práctico N° 2

**Redes de flujo, Clases de
Complejidad y Fuerza Bruta**

Grupo 3SAT

Profesor:
Ing. Victor Podberezky

11 de mayo de 2026

Índice

1. Introducción	2
2. Desarrollo del trabajo	2
2.1. La separación del Dojo	2
2.1.1. Explicación del problema	3
2.1.2. Propuesta de solución por red de flujos	3
2.1.3. Análisis de la solución	6
2.1.4. Ejemplo gráfico de ejecución de la solución	8
2.2. El despacho de los productos	10
2.2.1. Explicación del problema	11
2.2.2. Búsqueda exhaustiva sin poda	11
2.2.3. Propuesta	11
2.2.4. Explicación del funcionamiento	12
2.2.5. Función costo	12
2.2.6. Función límite	12
2.2.7. Complejidades	13
2.2.8. Ejemplo de ejecución	13
2.2.9. Pseudocódigo	14
2.3. Los centros de atención	16
3. Conclusión	16
4. Referencias	16

1. Introducción

El estudio avanzado de algoritmos permite no solo resolver problemas de manera eficiente, sino también comprender las limitaciones intrínsecas de la computación y la estructura subyacente de problemas complejos de optimización. Mientras que las técnicas de diseño clásicas proporcionan herramientas para construir soluciones, el análisis de flujos en redes y la clasificación de problemas según su complejidad permiten determinar la viabilidad de dichas soluciones en entornos reales.

En el presente informe, se profundizará en el estudio de tres ejes fundamentales de la algoritmia moderna: la modelización mediante redes de flujo, la implementación de estrategias de búsqueda exhaustiva y el marco teórico de las clases de complejidad. En la primera parte de este trabajo, se abordará la teoría de Redes de Flujo, una herramienta esencial para resolver problemas de transporte, logística y asignación de recursos. Se analizará cómo modelar un sistema de capacidades para determinar el flujo máximo posible entre nodos, aplicando conceptos como el teorema de Max-Flow Min-Cut.

Posteriormente, se explorará la técnica de Fuerza Bruta, evaluando su utilidad como punto de partida para la resolución de problemas combinatorios y discutiendo el costo computacional que implica la exploración completa del espacio de soluciones.

Finalmente, se presentará un análisis sobre las Clases de Complejidad, estableciendo las distinciones fundamentales entre problemas de las clases P , NP y NP -completos. Este apartado permitirá comprender la frontera entre lo tratable y lo intratable, brindando los criterios necesarios para identificar cuándo un problema exige una solución heurística o aproximada frente a la imposibilidad de hallar una solución exacta en tiempo polinomial.

2. Desarrollo del trabajo

2.1. La separación del Dojo

Las redes de flujo permiten modelar una gran variedad de problemas. Es posible representar cada arista dirigida como un conducto con capacidad limitada para transportar un fluido, y cada vértice como una intersección de dichos conductos. En consecuencia, por la ley de conservación, la cantidad de fluido que sale de cada intersección debe ser igual a la que ingresa. Gracias a

esta relación de equilibrio, es posible identificar características que permiten resolver problemas complejos mediante una reducción polinomial a una red de flujo.

El conocimiento sobre el comportamiento del flujo, el valor máximo que puede alcanzar la red y la identificación del “cuello de botella”, determinado por el corte que minimiza la capacidad entre la fuente y el sumidero, es lo que permite adaptar este modelo a problemas que, a simple vista, no parecen tener una estructura de red. De esta forma, se logran soluciones algorítmicas eficientes para desafíos de optimización y particionamiento.

2.1.1. Explicación del problema

Un dojo tiene “ n ” miembros, dos de ellos son profesores, cada uno de los miembros del dojo se relacionó al menos una vez con alguno de los profesores. Dado un grupo de relaciones R_A , cuyos elementos son tuplas del tipo $[(m_i, m_j), r_{ij}]$ donde m_i y m_j son miembros del dojo tal que $m_i \neq m_j$ y r_{ij} es un entero positivo representativo de las veces que se relacionó el miembro m_i con m_j , se busca dividir a los miembros del dojo entre los dos profesores de forma que la cantidad de veces que se relacionaron dos miembros de un grupo sea la máxima posible.

Supuestos del problema: Se supone que cada miembro del dojo se relacionó al menos una vez con alguno de los profesores, y que en el grupo de relaciones no pueden haber relaciones negativas. Además se supone que el primer profesor p_1 tiene número de miembro 1 y el profesor p_2 tiene número de miembro “ n ”.

2.1.2. Propuesta de solución por red de flujos

Dado el grupo de relaciones R_A y un grafo vacío G se crea un vertice por cada miembro m_i presente en R_A . Por cada relación (m_i, m_j) se crea una arista dirigida $m_i \rightarrow m_j$ de tal forma que $i < j$, con capacidad r_{ij} . La restricción “ $i < j$ ” genera que p_1 funcione como fuente (nodo s , del inglés *source*) y que p_2 funcione como sumidero de la red (nodo t , *terminal*), y que las aristas fluyan en el sentido $s \rightarrow t$.

Bajo la premisa de buscar que los grupos mantengan la máxima relación entre sus miembros, se busca la partición de esta red que minimice la suma de los pesos de las relaciones que se pierden en la separación.

Para lograrlo, en un primer paso de solución se calcula el flujo máximo de

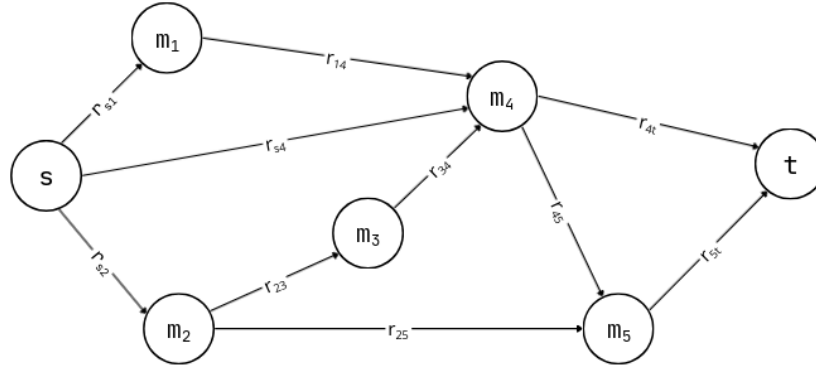


Figura 1: Ejemplo de red de flujo de relaciones

la red a través del Algoritmo de Edmonds-Karp. Por teorema fundamental de flujo se conoce que el valor de flujo máximo es igual a la capacidad de corte mínimo, donde el corte mínimo (S, T) es una partición de los nodos tal que $s \in S$ y $t \in T$. Estos subconjuntos S y T contienen aquellos nodos tal que en S se encuentran todos los nodos *alcanzables* por s en el grafo residual, es decir todos los nodos a los que se pueden llegar usando aristas de capacidad diferente de 0, y en T se encuentran los nodos que cumplen $T = G - \{S\}$.

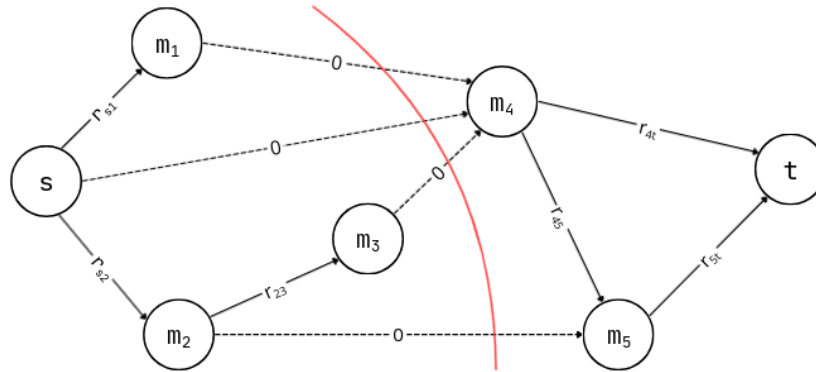


Figura 2: Corte mínimo de la red de relaciones

Una vez obtenidos los dos subconjuntos de G se comparan los tamaños de ambos para determinar cual de los dos grupos es el mayor y así poder retornar dicho grupo. Una posible implementación del código solución de esta propuesta puede ser el siguiente:

```

function definirNuevoRepresentante(miembros V, relaciones R,
    profesores P1, P2):
    G = Grafo new;
    for(relacion r = ((u, v), i) in R):
        if(u not in G.V):
            G.V.add(u);
        if(v not in G.V):
            G.V.add(v);
        G.E.add((u,v), i);

    s, t = P1, P2;
    G', _ = FlujoMaximo(G, s, t); # Esto genera el grafo
    residual.

    miembros_p1 = {s};
    cola = [s]
    visitados = {s}

    while(cola not empty):
        u = cola.pop(0)
        for(vecino v de u en G'):
            if(v not in visitados && G'.capacidad(u,v) > 0):
                visitados.add(v)
                miembros_p1.add(v)
                cola.add(v)

    miembros_p2 = G.V.notIn(miembros_p1)

    if(|miembros_p1| > |miembros_p2|):
        return P1;
    else if(|miembros_p1| < |miembros_p2|):
        return P2;
    else:
        return "Ambos profesores se dividen la misma cantidad
        de alumnos";

```

Listing 1: Pseudocódigo de la solución

Donde la función `FlujoMaximo` está determinada por el siguiente Pseudocódigo:

```

function FlujoMaximo(Grafo G, fuente s, sumidero t):
    for each(eje e in G):
        e.flujo = 0

    G_res = construirGrafoResidual(G)
    flujo_total = 0

    while(existeCamino(G_res, s, t)):
        P = G_res.shortestPath(s, t)

        w = P.getBottleneck()

        G.aplicarAumento(P, w)
        G_res.actualizarCapacidades(P, w)

        flujo_total += w

    return G_res, flujo_total

```

Listing 2: Pseudocódigo de FlujoMaximo

2.1.3. Análisis de la solución

A partir del pseudocódigo presentado anterior se puede realizar un análisis en profundidad de la solución propuesta, comenzando por un análisis de uso de recursos de tiempo y espacio, para luego terminar de definir detalles que pueden haber quedado sin analizar.

Análisis de eficiencia espacial: En cuestiones de uso de espacio, este algoritmo ocupa distintos ordenes de complejidad según como se implemente el grafo a utilizarse. En caso de usar una lista de adyacencia, la complejidad espacial de guardar el grafo de relaciones conllevaría $O(V + E)$, mientras que una matriz de adyacencias utiliza $O(V^2)$. Dado que en este trabajo se implementaron las soluciones algorítmicas en el lenguaje de programación Python, se utilizó la lista de adyacencias¹ y por lo tanto, en consecuencia a este hecho, la complejidad espacial del algoritmo será igual a $O(V + E)$. Cabe aclararse adicionalmente, que la operatoria de búsqueda de flujo máximo requiere $O(V)$ espacio adicional, que por sumarse a la complejidad mencionada anteriormente no afecta al requerimiento espacial final ya que $O(V) \subset O(V + E)$.

¹En este caso sería más adecuado decir “Diccionario de adyacencias”.

Análisis de eficiencia temporal: Respecto al tiempo de ejecución del algoritmo, construir la red de relaciones lleva $O(V + E)$ en ordenes de tiempo e identificar los distintos subconjuntos tambien requieren $O(V + E)$. Dado que se eligió el algoritmo de Edmonds-Karp, este tiene una complejidad de $O(V + E^2)$ dado que iterando por cada arista ($O(E)$) se busca un camino de aumento ($O(V + E)$) lo que resulta en la complejidad dada. Adicionalmente, contabilizar cuantos miembros contienen cada subconjuntos puede conllevar como mucho una complejidad de $O(V)$.

Ya que $O(V) \subset O(V + E)$ y $O(V + E) \subset O(V + E^2)$, la complejidad resulta:

$$2O(V + E) + O(V + E^2) + O(V) = O(V + E^2) \quad (1)$$

Otros detalles de la solución: En esta subsección del análisis de la propuesta algorítmica se busca responder a las preguntas “¿*Realmente la propuesta predicirá la división final?*” y “¿*Puede una misma instancia tener diferentes soluciones?* ¿*Que implicación tiene elegir una solución por encima de otra?*”.

Respecto a la primer pregunta, esta solución no predicirá la división final entre los miembros, sino que esta solución presenta un modelo de estimación simplificada. Esto está dado porque el corte mínimo asume que el sistema busca el estado de menor fricción, es decir la separación del grupo de miembros del dojo que mantenga los vinculos más fuertes. Adicionalmente, dado que es un modelo estimativo, puede ocurrir en la realidad que por fuera de toda estimación dada por esta solución, un alumno elija irse con un profesor en vez del estimado. Esta variación además podría generar un efecto cascada de cambio de profesor de alumnos de relación fuerte con el miembro que eligió al otro profesor.

Otra característica de esta solución es el hecho de que pueden haber múltiples ópciones de corte en una misma instancia. En una red de flujo, el flujo máximo toma un único valor, pero el corte mínimo de capacidades de dicha red no tiene porque ser uno solo. Tomando el caso donde hay un solo miembro además de ambos profesores y que dicho miembro interactuó una misma cantidad de veces con ambos profesores, ocurre el caso donde la red de flujo tendrá un flujo máximo igual al número de veces que haya interactuado el alumno con los profesores pero el corte que minimiza las capacidades puede realizarse desde la fuente o desde el sumidero, por lo que dará distintas soluciones según como se haya determinado la operación de corte.

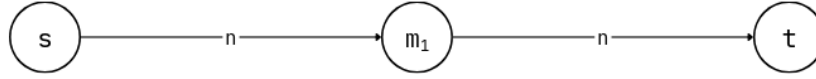


Figura 3: Caso borde donde se pone en duda donde se realizará el corte mínimo

Este problema de incertidumbre es equivalente para cualquier cantidad de miembros, siempre que se cumpla que ambos profesores hayan interactuado la misma cantidad de veces con ellos y no hayan interacciones entre miembros. Consecuentemente, si bien el Teorema de Flujo Máximo-Corte Mínimo ofrece una solución óptima en términos de capacidad, la existencia de múltiples cortes mínimos sugiere que ciertos miembros se encuentran en una posición de equilibrio social. La elección de la fuente (s_1) para iniciar la exploración del grafo residual inclina la asignación de estos miembros en disputa hacia el primer profesor.

2.1.4. Ejemplo gráfico de ejecución de la solución

En esta sección se mostrará la solución de una instancia del problema según el algoritmo mostrado anteriormente. Dada la instancia siguiente instancia:

$[(1, 2], 4), ([1, 3], 4), ([1, 5], 1), ([2, 5], 1), ([5, 7], 5), ([3, 4], 4), ([5, 6], 5), ([6, 7], 5)]$

Con el profesor 1 representado por el número 1 y el profesor 2 representado por el número 7. Se construye una red de flujo a partir de ella, donde los nodos fuente y sumidero son el profesor 1 y el profesor 2 respectivamente, los nodos intermedios son los miembros del dojo y las aristas dirigidas y ponderadas son las veces que interactuaron dos miembros. La instancia elegida para este ejemplo conforma la red mostrada en la Figura 4.

A partir de esta red, se busca el corte que minimice las capacidades de las aristas. Como se muestra en la Figura 5, el corte mínimo logra separar la red en dos partes.

Es así como a partir del grafo residual de la red pueden conformarse dos

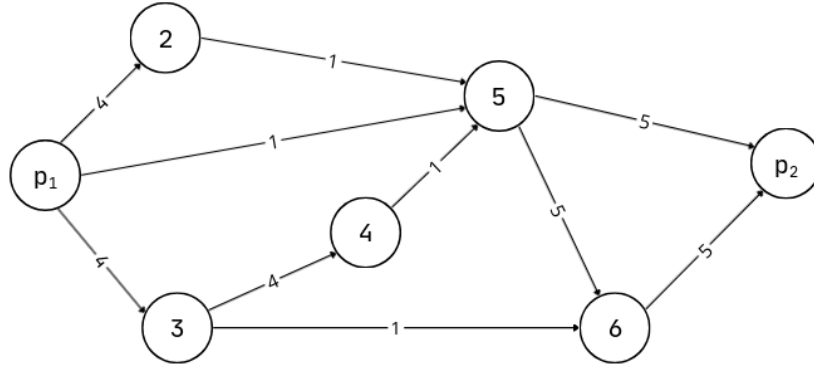


Figura 4: Red conformada por la instancia dada.

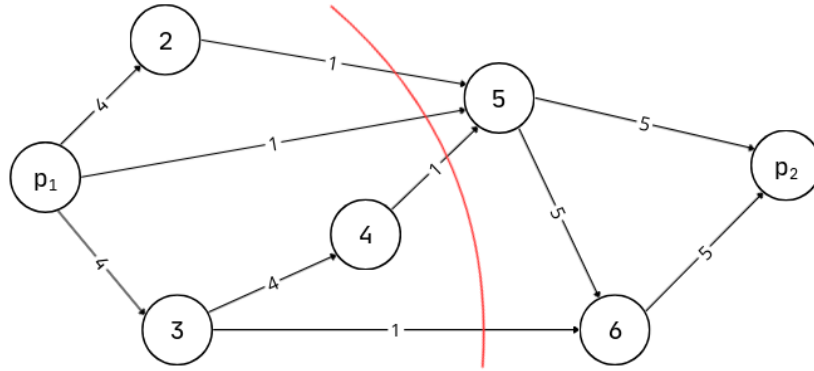


Figura 5: Corte mínimo de la instancia.

grupos de miembros y estos siempre tendrán la máxima relación entre sus pares. Así que tomando el grafo residual de la red se buscan todos los nodos “alcanzables” por el profesor 1 y se conforma el primero de los grupos.

Por último, usando el grupo recientemente formado se buscan aquellos miembros que no conformen dicho grupo para encontrar el grupo de miembros que seleccionarían al profesor 2. Así formando los dos grupos solución del algoritmo tal como se muestra en la figura 7.

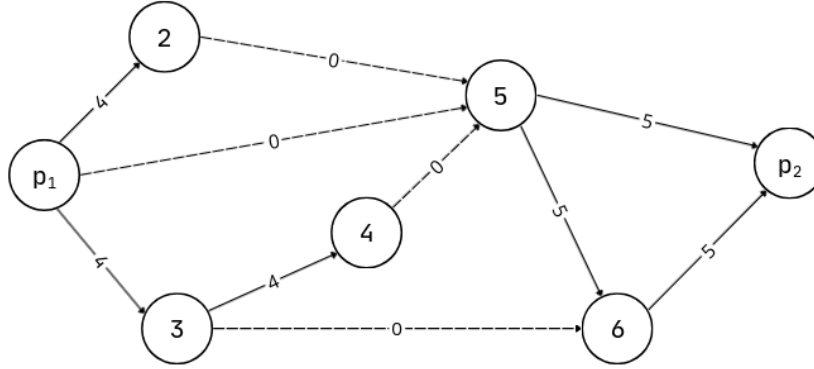


Figura 6: Grafo residual de la red de flujos de la instancia.

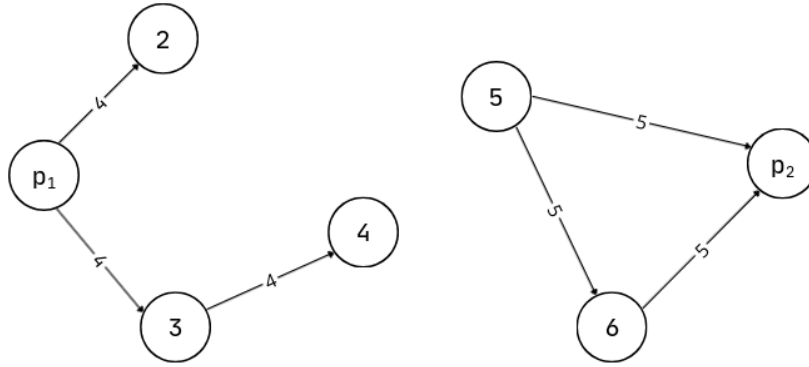


Figura 7: Grupos solución a la instancia.

2.2. El despacho de los productos

En muchos problemas de logística y optimización de recursos no solo importa maximizar una ganancia, sino también minimizar las pérdidas asociadas al desperdicio de capacidad o recursos no utilizados. Este tipo de situaciones aparece frecuentemente en problemas de empaquetado, transporte y asignación de tareas, donde encontrar una solución óptima requiere analizar múltiples configuraciones posibles.

En esta sección se resolverá un problema de optimización mediante la técnica de *Branch & Bound*. Esta técnica puede entenderse como una mejora sobre la búsqueda exhaustiva tradicional, ya que mantiene la optimalidad de fuerza bruta, pero reduce considerablemente la cantidad de soluciones analizadas gracias al uso de podas.

2.2.1. Explicación del problema

Se tiene un conjunto de productos donde cada producto i está representado por la tupla (s_i, v_i) , donde s_i representa el tamaño del producto y v_i el valor obtenido por enviarlo. Además, se tiene k contenedores iguales, cada uno con capacidad máxima C , y un factor de penalización a aplicado sobre el espacio libre de los contenedores que hayan sido utilizados. Cada producto puede colocarse en un único contenedor o no enviarse. A su vez, la suma de los tamaños de los productos que se encuentran dentro de un mismo contenedor no puede superar su capacidad máxima.

El objetivo consiste en maximizar la ganancia total obtenida por el envío de productos, teniendo en cuenta además la penalización por espacio desaprovechado. Esto está dado por:

$$G = \sum v_i - a \cdot \sum \text{espacio_libre}$$

2.2.2. Búsqueda exhaustiva sin poda

Una solución al problema consiste en realizar una búsqueda exhaustiva de todas las configuraciones posibles. Para cada producto existen $k+1$ decisiones posibles, ya que este puede colocarse en cualquiera de los k contenedores disponibles o directamente no enviarse. Por lo tanto, si existen n productos, el número total de combinaciones posibles que podrían generarse es:

$$(k + 1)^n$$

Esta manera garantiza encontrar la solución óptima ya que analiza todas las combinaciones válidas posibles. Sin embargo, es ineficiente debido a que incluso las ramas que claramente no superan la mejor solución encontrada continúan explorándose hasta el final. Para evitar este problema, se agrega una función límite que permite estimar si una rama todavía tiene posibilidades de mejorar la solución óptima actual y, en caso contrario, descartarla antes de recorrerla completamente.

2.2.3. Propuesta

Se propone construir un árbol de decisiones donde cada nivel representa un producto distinto. Para cada producto i se generan hasta $k + 1$ ramas

posibles, correspondientes a colocarlo en alguno de los k contenedores disponibles o directamente no enviarlo. De esta manera, cada nodo del árbol mantiene el estado parcial de la solución construida hasta el momento.

Durante el recorrido del árbol, el algoritmo calcula una estimación del máximo beneficio que todavía podría alcanzarse usando los productos restantes. Si esta no supera la mejor solución encontrada hasta el momento, entonces la rama actual se poda y deja de explorarse, evitando así recorrer nodos que no conducen a una solución óptima.

2.2.4. Explicación del funcionamiento

El algoritmo analiza los productos uno por uno y, para cada uno de ellos, intenta colocarlo en todos los contenedores donde exista espacio suficiente. Después de cada asignación posible, se actualiza la ganancia parcial obtenida y se calcula la función límite de la nueva rama generada. La exploración continúa solamente si la rama todavía tiene posibilidades de superar la mejor solución encontrada hasta ese momento. Además, también se considera la posibilidad de no enviar el producto actual, generando así otra rama dentro del árbol de búsqueda.

Cuando se alcanza una hoja del árbol, es decir, cuando ya se tomaron decisiones para todos los productos, se calcula la penalización total asociada al espacio libre de los contenedores utilizados y se obtiene la ganancia final de la configuración construida. Finalmente, si esta ganancia resulta mayor que la mejor solución conocida, la solución se actualiza.

2.2.5. Función costo

Representa la ganancia obtenida para un estado dado.

$$G = \sum v_i - a \cdot \sum (C - U_j)$$

Donde U_j es el espacio utilizado en el contenedor “ j ”

2.2.6. Función límite

Permite estimar el máximo beneficio posible que podría alcanzarse.

$$cota_superior = ganancia_actual + \sum valores_restantes$$

Esta función representa una estimación optimista, ya que considera que todos los productos restantes podrían agregarse sin restricciones de capacidad.

$$cota_superior \leq mejor_solucion$$

2.2.7. Complejidades

Para cada producto existen $k + 1$ decisiones posibles, por lo tanto, en el peor caso, el algoritmo puede recorrer:

$$(k + 1)^n$$

Dando una complejidad temporal:

$$T(n) = O((k + 1)^n)$$

Teniendo en cuenta que la profundidad máxima del árbol es igual a la cantidad de productos n , y que las estructuras auxiliares que usamos dependen de la cantidad de contenedores k , la complejidad espacial total del algoritmo queda dada por:

$$E(n) = O(n + k)$$

2.2.8. Ejemplo de ejecución

Teniendo el siguiente conjunto de productos:

```
productos = [(2,10), (5,10), (7,20)]  
k = 2  
C = 8  
a = 5
```

Asignamos los productos 0 y 1 al primer contenedor, mientras que el producto 2 lo colocamos en el segundo contenedor:

```
Contenedor 1 = [0,1]  
Contenedor 2 = [2]
```

De esta manera, los productos del primer contenedor ocupan $2 + 5 = 7$, mientras que la del segundo contenedor 7. Por lo tanto, ambos contenedores tienen 1 unidad de espacio libre.

La penalización total aplicada sobre el espacio no usado es:

$$5(1 + 1) = 10$$

Por otro lado, el valor total obtenido por enviar los productos es:

$$10 + 10 + 20 = 40$$

Finalmente, la ganancia total es de:

$$40 - 10 = 30$$

Por lo tanto, el algoritmo retorna:

$$(30, [[0, 1], [2]])$$

2.2.9. Pseudocódigo

```
main(productos, k, C, a):  
  
    mejor_ganancia = -infinito  
    mejor_solucion = []  
    contenedores = arreglo de k listas vacias  
    ocupacion = arreglo de k ceros  
  
    branch_and_bound(i = 0, productos, contenedores,  
                     ocupacion, ganancia_actual = 0)  
  
    retornar mejor_ganancia, mejor_solucion  
  
branch_and_bound(i, productos, contenedores, ocupacion,  
ganancia_actual):  
  
    if(i == cantidad_productos):  
  
        calcular penalizacion total  
  
        ganancia_final = ganancia_actual - penalizacion  
  
        if(ganancia_final > mejor_ganancia):
```

```

        actualizar mejor_ganancia
        guardar copia de contenedores

    retornar

cota = ganancia_actual + suma_valores_restantes(i)

if(cota <= mejor_ganancia):

    podar rama
    retornar

cantidad, valor = productos[i]

para cada contenedor j:

    if(ocupacion[j] + cantidad <= C):

        agregar producto i al contenedor j

        ocupacion[j] += cantidad

        branch_and_bound(i + 1, productos, contenedores,
                           ocupacion, ganancia_actual +
valor)

        deshacer cambios

    explorar no enviar el producto actual

```

Para este problema se utilizó la técnica de *Branch & Bound* para resolverlo. Se partió de una solución de búsqueda exhaustiva tradicional que garantiza encontrar el óptimo pero requiere recorrer todas las configuraciones posibles. Sobre esta idea se construyó una mejora mediante el uso de funciones límite y podas, permitiendo descartar ramas que no pueden superar la mejor solución encontrada.

2.3. Los centros de atención

3. Conclusión

A partir de este trabajo de investigación y de la puesta en práctica de los conocimientos obtenidos en esta etapa avanzada de la Teoría de Algoritmos, se ha logrado profundizar en el análisis de problemas de optimización de alta complejidad. El estudio de las Redes de Flujo permitió comprender la modelización de sistemas con restricciones de capacidad, mientras que el abordaje de la Fuerza Bruta y las Clases de Complejidad brindó una perspectiva integral sobre los límites de la eficiencia computacional y la distinción entre problemas tratables e intratables.

Asimismo, se enriqueció la labor realizada mediante una investigación exhaustiva que integró tanto el material bibliográfico de la cátedra como el análisis de publicaciones técnicas y recursos académicos complementarios. Esta base teórica fue fundamental para entender no solo el funcionamiento de los algoritmos, sino también las implicancias de las reducciones entre problemas y la jerarquía de complejidad que rige a la informática moderna.

Finalmente, la implementación en el lenguaje Python de las soluciones propuestas para redes de flujo y algoritmos de búsqueda exhaustiva resultó clave para validar los modelos teóricos. Este proceso permitió observar el comportamiento real de los algoritmos frente a diferentes volúmenes de datos y casos de prueba, consolidando la transición entre el diseño conceptual y la aplicación práctica de soluciones óptimas en entornos de ingeniería.

4. Referencias